

HW 6: Study Gini Index

Instructions

- Study Gini Index
- Compute the toy example using Gini Index
- Change criterion in the imported library, using Gini Index
- Compare Gini Index vs Entropy
- Use another dataset (datasets.csv)
- Play with parameters:
 - max_depth
 - min_samples_split
 - min_samples_leaf
- Explain your understanding after trying these different parameters

```
In [4]: import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

```
In [5]: toy_df = pd.read_csv("toy_data.csv")
df = pd.read_csv("dataset.csv")
```

```
In [42]: toy_df
```

Out[42]:

	age	income	student	credit rating	buys computer
0	<=30	high	no	fair	no
1	<=30	high	no	excellent	no
2	31-40	high	no	fair	yes
3	>40	medium	no	fair	yes
4	>40	low	yes	fair	yes
5	>40	low	yes	excellent	no
6	31-40	low	yes	excellent	yes
7	<=30	medium	no	fair	no
8	<=30	low	yes	fair	yes
9	>40	medium	yes	fair	yes
10	<=30	medium	yes	excellent	yes
11	31-40	medium	no	excellent	yes
12	31-40	high	yes	fair	yes
13	>40	medium	no	excellent	no

```
In [43]: df
```

Out[43]:

	feature_0	feature_1	feature_2	feature_3	feature_4	target
0	0.374540	0.950714	0.731994	0.598658	0.156019	0
1	0.155995	0.058084	0.866176	0.601115	0.708073	1
2	0.020584	0.969910	0.832443	0.212339	0.181825	1
3	0.183405	0.304242	0.524756	0.431945	0.291229	0
4	0.611853	0.139494	0.292145	0.366362	0.456070	1
...	...	...	...	...	...	...
145	0.841829	0.139772	0.795267	0.201627	0.163656	1
146	0.164266	0.814575	0.665197	0.523065	0.358830	1
147	0.877201	0.392445	0.816599	0.439135	0.376944	1
148	0.462680	0.301378	0.747609	0.502720	0.232213	0
149	0.899575	0.383891	0.543553	0.906472	0.624238	1

150 rows × 6 columns

```
In [1]: import math
```

```
In [6]: def calEntropy(col):
    en = 0
    dictJ = col.value_counts().to_dict()
    n = len(col)
    for i in dictJ.values():
        en -= math.log2(i/n)*(i/n)
    return en

calEntropy(toy_df["buys computer"])
```

```
Out[6]: 0.9402859586706311
```

```
In [7]: def calGini(col):
    gn = 1
    dictJ = col.value_counts().to_dict()
    n = len(col)
    for i in dictJ.values():
        gn -= (i/n) ** 2
    return gn
```

```
calGini(toy_df["buys computer"])
```

Out[7]: 0.4591836734693877

```
In [44]: label_encoder = LabelEncoder()
toy_df['age'] = label_encoder.fit_transform(toy_df['age'])
toy_df['income'] = label_encoder.fit_transform(toy_df['income'])
toy_df['student'] = label_encoder.fit_transform(toy_df['student'])
toy_df['credit rating'] = label_encoder.fit_transform(toy_df['credit rating'])
toy_df['buys computer'] = label_encoder.fit_transform(toy_df['buys computer'])

X = toy_df.drop('buys computer', axis=1)
y = toy_df['buys computer']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

clf.fit(X_train, y_train)
```

Out[44]:

DecisionTreeClassifier ⓘ ⓘ

Parameters

```
In [45]: y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

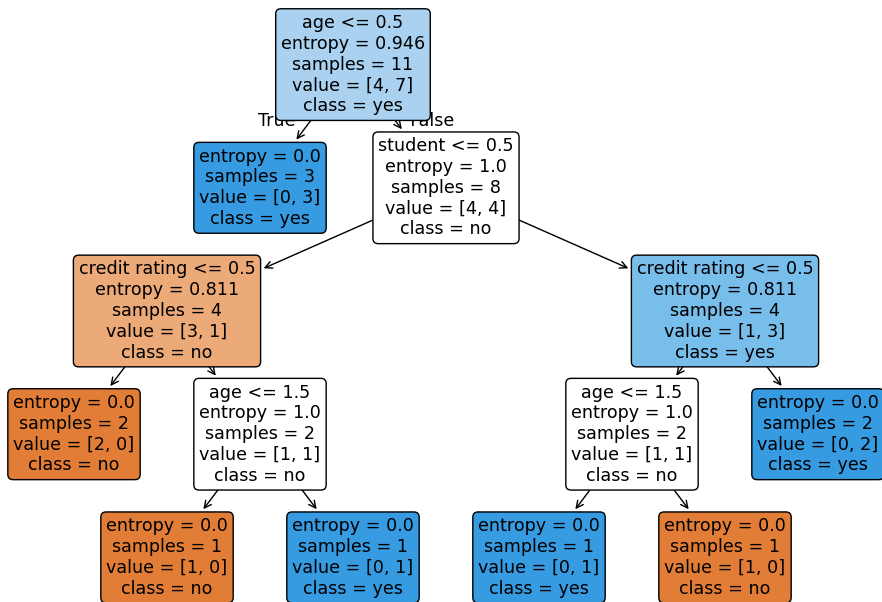
Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Confusion Matrix:

```
[[1 0]
 [0 2]]
```



```
In [46]: clf2 = DecisionTreeClassifier(criterion='gini', random_state=42)

clf2.fit(X_train, y_train)
```

Out[46]:

DecisionTreeClassifier ⓘ ⓘ

Parameters

```
In [47]: y_pred = clf2.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf2, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

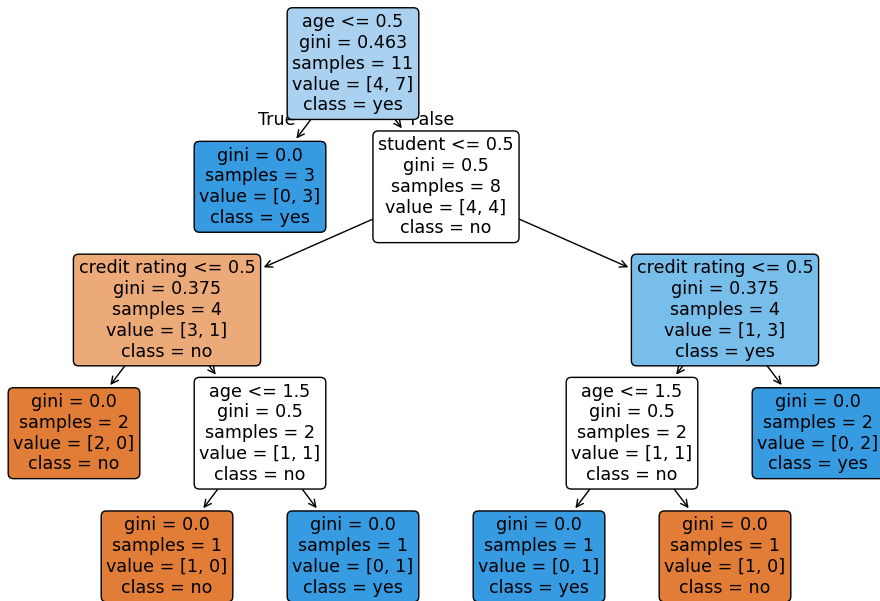
Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Confusion Matrix:

```
[[1 0]
 [0 2]]
```



In toy dataset the decision tree have no different so we should test it on larger data.

```
In [48]: X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [49]: clf = DecisionTreeClassifier(criterion='entropy', random_state=42)
```

```
clf.fit(X_train, y_train)
```

```
Out[49]: + DecisionTreeClassifier ⓘ ⓘ
          ▶ Parameters
```

```
In [50]: y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

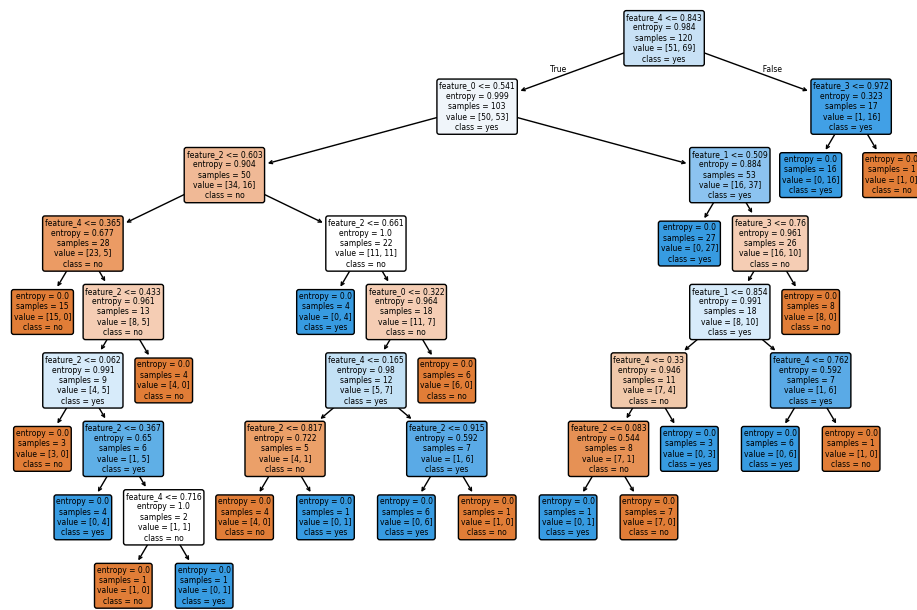
Accuracy: 0.67

Classification Report:

	precision	recall	f1-score	support
0	0.62	0.42	0.50	12
1	0.68	0.83	0.75	18
accuracy			0.67	30
macro avg	0.65	0.62	0.62	30
weighted avg	0.66	0.67	0.65	30

Confusion Matrix:

```
[[ 5  7]
 [ 3 15]]
```



```
In [51]: clf2 = DecisionTreeClassifier(criterion='gini', random_state=42)
```

```
clf2.fit(X_train, y_train)
```

```
Out[51]: DecisionTreeClassifier
Parameters
```

```
In [52]: y_pred = clf2.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf2, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

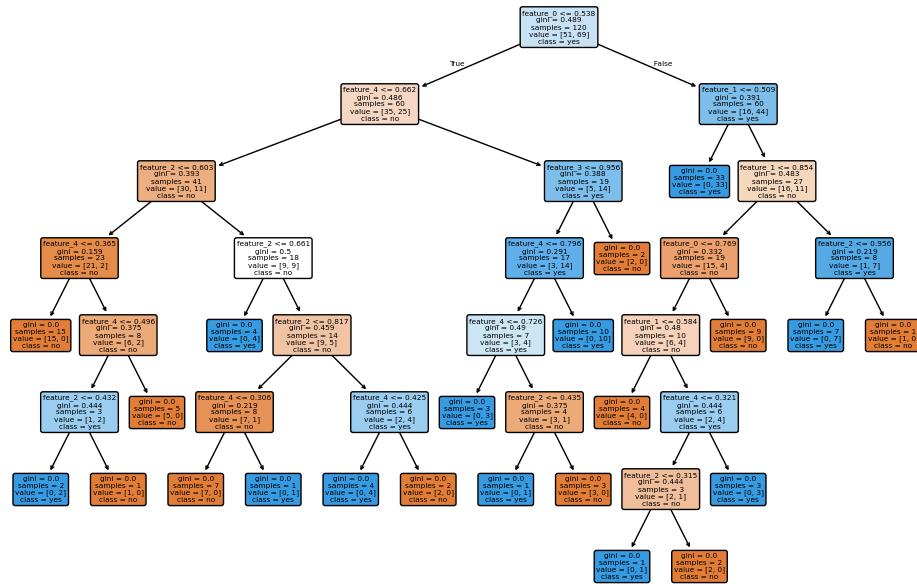
Accuracy: 0.80

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.75	0.75	12
1	0.83	0.83	0.83	18
accuracy			0.80	30
macro avg	0.79	0.79	0.79	30
weighted avg	0.80	0.80	0.80	30

Confusion Matrix:

```
[[ 9  3]
 [ 3 15]]
```



In this dataset Entropy criterion have only 0.67 accuracy and 8 level height while Gini criterion have 0.80 accuracy and 7 level height.

this time we are going to adjust parameters.

```
In [53]: clf = DecisionTreeClassifier(criterion='entropy', random_state=42, max_depth=6)
```

```
clf.fit(X_train, y_train)
```

```
Out[53]: DecisionTreeClassifier
Parameters
```

```
In [54]: y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

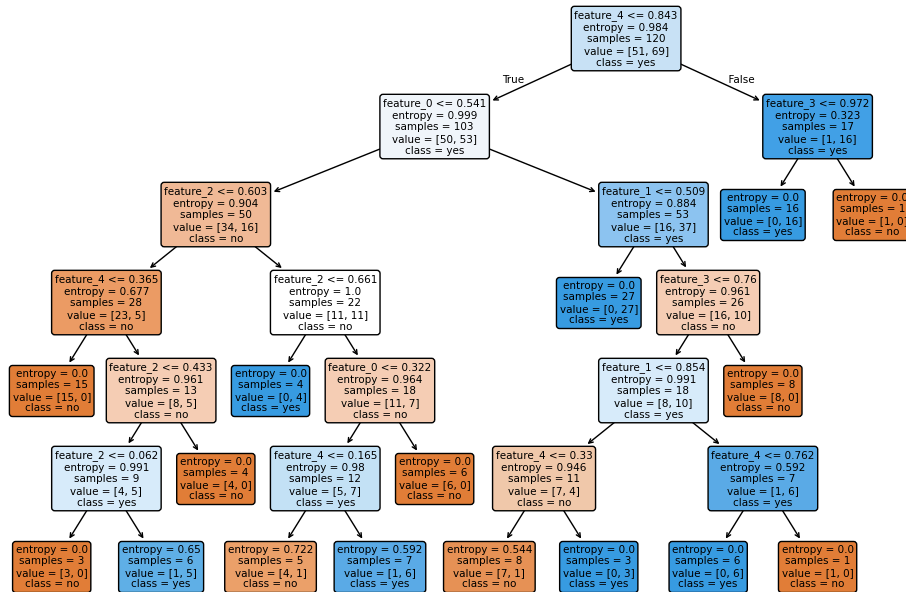
Accuracy: 0.60

Classification Report:

	precision	recall	f1-score	support
0	0.50	0.25	0.33	12
1	0.62	0.83	0.71	18
accuracy			0.60	30
macro avg	0.56	0.54	0.52	30
weighted avg	0.57	0.60	0.56	30

Confusion Matrix:

```
[[ 3  9]
 [ 3 15]]
```



Max depth parameter is to restrict the max level of tree.

```
In [55]: clf = DecisionTreeClassifier(criterion='entropy', random_state=42, min_samples_split=4)
```

```
clf.fit(X_train, y_train)
```

```
Out[55]: DecisionTreeClassifier
Parameters
```

```
In [56]: y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

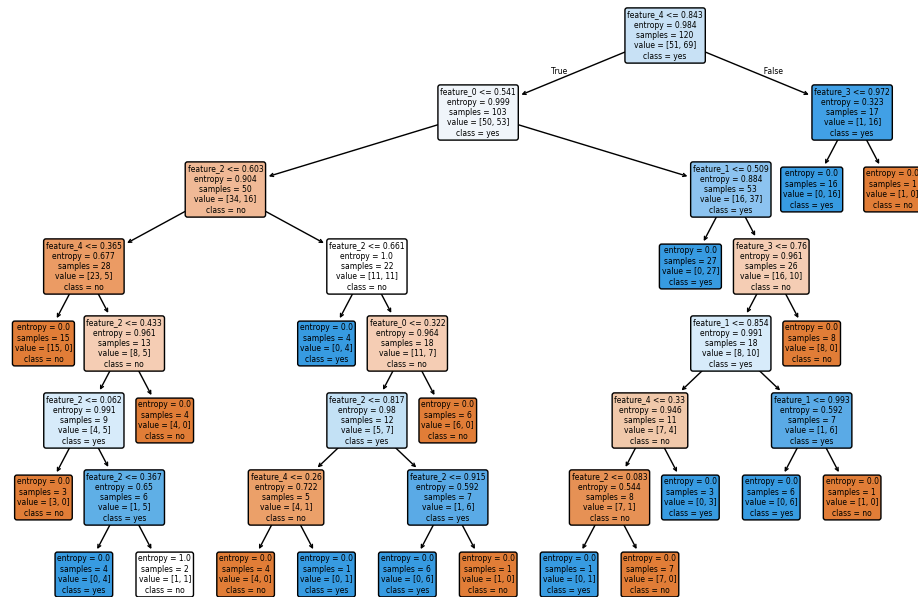
Accuracy: 0.67

Classification Report:

	precision	recall	f1-score	support
0	0.62	0.42	0.50	12
1	0.68	0.83	0.75	18
accuracy			0.67	30
macro avg	0.65	0.62	0.62	30
weighted avg	0.66	0.67	0.65	30

Confusion Matrix:

```
[[ 5  7]
 [ 3 15]]
```



min_samples_split parameter is to branching only node with samples more than or equal makes the tree eager to split tiny group data.

```
In [57]: clf = DecisionTreeClassifier(criterion='entropy', random_state=42, min_samples_leaf=3)
```

```
clf.fit(X_train, y_train)
```

```
Out[57]:
```

```
+ DecisionTreeClassifier ⓘ ⓘ
  Parameters
```

```
In [58]: y_pred = clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

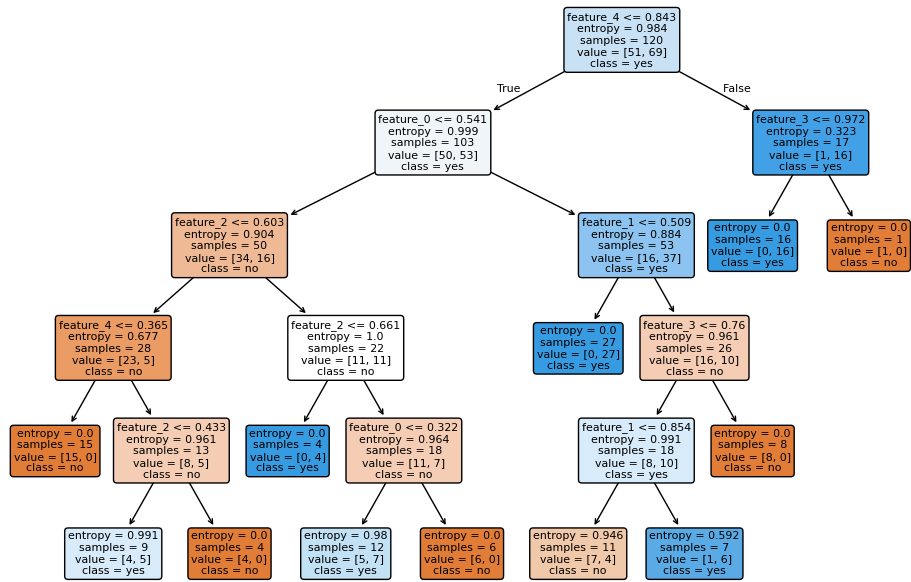
Accuracy: 0.53

Classification Report:

		precision	recall	f1-score	support
	0	0.38	0.25	0.30	12
	1	0.59	0.72	0.65	18
	accuracy			0.53	30
	macro avg	0.48	0.49	0.47	30
	weighted avg	0.50	0.53	0.51	30

Confusion Matrix:

```
[[ 3  9]
 [ 5 13]]
```

With max_depth 5 and min_samples_split = 10 we can improve accuracy from 0.67 to 0.70.